

UNIT – 2 : Design And Analysis of Machine Learning Algorithms

Guidelines for machine learning experiments:

1. Define the Problem

- Clarify Objectives: Clearly outline what you want to achieve (e.g., classification, regression).
- Identify Metrics: Choose appropriate metrics (e.g., accuracy, precision, recall, F1 score) based on the problem type.

2. Data Preparation

- Data Collection: Gather data relevant to the problem.
- Data Cleaning: Handle missing values, outliers, and inconsistencies.
- Feature Engineering: Create new features that may improve model performance.
- Data Splitting: Split data into training, validation, and test sets to evaluate model performance accurately.

3. Model Selection

- Baseline Model: Start with a simple model to set a performance baseline.
- Algorithm Choice: Experiment with different algorithms (e.g., decision trees, SVM, neural networks) based on the problem and data characteristics.

4. Experiment Design

- Hyperparameter Tuning: Use techniques like grid search or random search to find optimal parameters.
- Cross-Validation: Employ k-fold cross-validation to ensure the model's robustness.

5. Implementation

- Reproducibility: Use version control for code and track data versions.
- Documentation: Keep thorough records of experiments, including configurations and results.

6. Evaluation

- Test Set Evaluation: Use the test set only for the final evaluation to avoid bias.
- Analyze Results: Look at performance metrics, confusion matrices, and ROC curves to understand model strengths and weaknesses.

7. Iteration

- **Refinement:** Based on evaluation, iterate on model design, feature selection, and hyperparameter tuning.
- **Experiment Tracking:** Use tools like MLflow or TensorBoard to track experiments and visualize results.

8. Deployment and Monitoring

- **Model Deployment:** Once satisfied with the model, deploy it into a production environment.
- **Monitoring:** Continuously monitor model performance in production to detect drifts or degradation over time.

9. Ethical Considerations

- **Bias and Fairness:** Assess models for potential biases and ensure fairness across different demographic groups.
- **Transparency:** Maintain transparency about the model's limitations and the data used.

10. Continuous Learning

- **Stay Updated:** Keep up with the latest research and techniques in machine learning.
- **Feedback Loop:** Use feedback from model performance to inform future experiments and improve methodologies.

Cross Validation (CV) and resampling :

Cross-Validation :

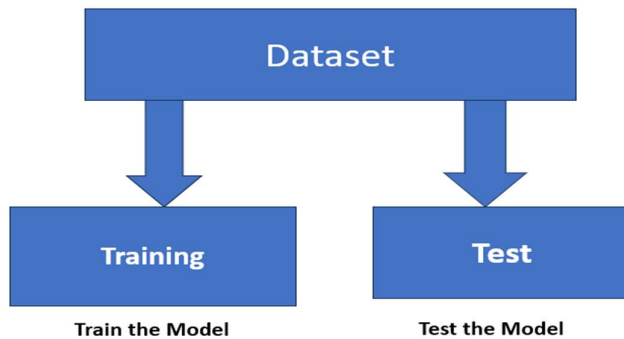
Cross-Validation (CV) is a technique used to evaluate how well a machine learning model performs. It works by dividing the dataset into several smaller parts (subsets). Here's how it generally works:

1. **Divide the Data:** The dataset is split into multiple subsets (or folds).
2. **Training and Testing:** The model is trained on some of these subsets and tested on the remaining ones. This is repeated multiple times, each time using different subsets for training and testing.
3. **Performance Assessment:** By averaging the results from these tests, we get a better estimate of how the model will perform on new, unseen data.

2. Purpose of Cross-Validation

- **Model Evaluation:** It gives a better estimate of how the model will perform on new, unseen data.

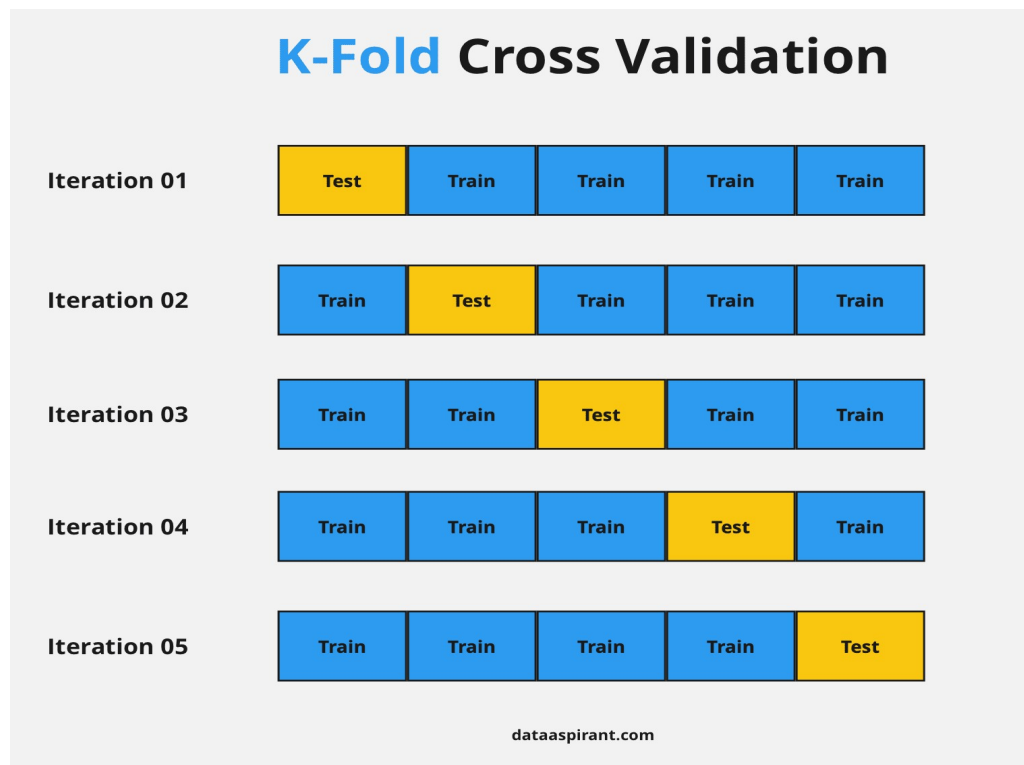
- **Prevent Overfitting:** By using different parts of the data for training and testing, it reduces the chance of the model fitting too closely to the training data.



3. Types of Cross-Validation

a. K-Fold Cross-Validation:

- The data is divided into k equal parts (or folds).
- The model is trained on k-1 parts and tested on the remaining part. This is repeated k times, each time using a different part for testing.
- Example: If k=5, the data is split into 5 parts, and the model is tested on each part one at a time.



Example:

Suppose you have a dataset with 800 data points, and you choose $K = 5$ (5-fold cross-validation).

The dataset is split into 5 folds, each containing 160 data points. The process repeats 5 iterations. In each iteration, the model is trained on 4 folds (640 samples) and validated on the 1 remaining fold (160 samples). After **cross-validation**, the **performance** on all 5 iterations is averaged.

Example of K-Fold Cross-Validation

Dataset: Let's say we have a dataset with 10 samples, labeled from 1 to 10.

K-Fold Cross-Validation Steps:

1. **Choose K:** Let's set $K=5$. This means we will divide our dataset into 5 folds.
 2. **Divide the Data:** We split the dataset into 5 folds like this:
 - **Fold 1:** [1, 2]
 - **Fold 2:** [3, 4]
 - **Fold 3:** [5, 6]
 - **Fold 4:** [7, 8]
 - **Fold 5:** [9, 10]
 3. **Training and Testing:**
 - **Iteration 1:**
 - **Training Set:** [3, 4, 5, 6, 7, 8, 9, 10] (all folds except Fold 1)
 - **Test Set:** [1, 2] (Fold 1)
 - **Iteration 2:**
 - **Training Set:** [1, 2, 5, 6, 7, 8, 9, 10]
 - **Test Set:** [3, 4] (Fold 2)
 - **Iteration 3:**
 - **Training Set:** [1, 2, 3, 4, 7, 8, 9, 10]
 - **Test Set:** [5, 6] (Fold 3)
 - **Iteration 4:**
 - **Training Set:** [1, 2, 3, 4, 5, 6, 9, 10]
 - **Test Set:** [7, 8] (Fold 4)
 - **Iteration 5:**

- **Training Set:** [1, 2, 3, 4, 5, 6, 7, 8]
 - **Test Set:** [9, 10] (Fold 5)
4. **Evaluate Model:** After each iteration, we evaluate the model's performance on the test set.
 5. **Average Performance:** Once all 5 iterations are complete, we average the performance metrics (like accuracy, precision, etc.) from each iteration to get the final model performance.

Summary

K-Fold Cross-Validation helps ensure that the model is tested on different subsets of the data, providing a robust estimate of its performance and helping to avoid overfitting.

b. Stratified K-Fold Cross-Validation:

- Similar to k-fold but keeps the same proportion of classes in each fold.
- This is useful when the classes are imbalanced (e.g., many more examples of one class than another).
- **Example:** If 70% of your data is one class and 30% another, each fold will maintain this ratio.

Example Dataset

Let's consider a dataset with 6 samples:

- **Samples:** [A, B, C, D, E, F]
- **Classes:** [1, 1, 2, 2, 3, 3]

Class Distribution:

- Class 1: 2 samples (A, B)
- Class 2: 2 samples (C, D)
- Class 3: 2 samples (E, F)

Stratified K-Fold Cross-Validation (K=3)

1. **Choose K:** We will use K=3, which means we will create 3 folds.
2. **Divide the Data into Folds:**
 - We want to keep the class distribution similar in each fold.

Folds:

- **Fold 1:** [A (1), C (2)]
- **Fold 2:** [B (1), D (2)]

- **Fold 3:** [E (3), F (3)]

Training and Testing

Now, let's perform the training and testing:

1. Iteration 1:

- **Training Set:** [B, C, D, E, F] (Folds 2, 3)
- **Test Set:** [A] (Fold 1)

2. Iteration 2:

- **Training Set:** [A, C, D, E, F] (Folds 1, 3)
- **Test Set:** [B] (Fold 2)

3. Iteration 3:

- **Training Set:** [A, B, C, D] (Folds 1, 2)
- **Test Set:** [E, F] (Fold 3)

Summary

- **Class Representation:** Each fold maintains the representation of classes. For instance:
 - Fold 1 has Class 1 and Class 2.
 - Fold 2 has Class 1 and Class 2.
 - Fold 3 has Class 3.

By ensuring that each fold has samples from all classes, we get a more reliable estimate of the model's performance.

C. Leave-One-Out Cross-Validation (LOOCV):

- Each data point is used once as a test set while the rest are used for training.
- **Example:** For 100 samples, the model is trained 100 times, each time leaving out one different sample for testing.

Example Dataset

Let's say we have a small dataset with 5 samples:

- **Samples:** [A, B, C, D, E]
- **Classes:** [1, 1, 2, 2, 3]

Leave-One-Out Cross-Validation (LOOCV)

In LOOCV, we use each sample in the dataset as a test set once, while the rest serve as the training set.

Steps:

1. **Total Samples:** We have 5 samples (A, B, C, D, E).
2. **Iterations:** We will perform 5 iterations, each time leaving out one sample for testing.

Iterations:

1. **Iteration 1:**
 - **Training Set:** [B, C, D, E]
 - **Test Set:** [A]
2. **Iteration 2:**
 - **Training Set:** [A, C, D, E]
 - **Test Set:** [B]
3. **Iteration 3:**
 - **Training Set:** [A, B, D, E]
 - **Test Set:** [C]
4. **Iteration 4:**
 - **Training Set:** [A, B, C, E]
 - **Test Set:** [D]
5. **Iteration 5:**
 - **Training Set:** [A, B, C, D]
 - **Test Set:** [E]

Summary

In LOOCV, each sample is used as a test set exactly once, while all other samples are used for training. This method allows for a thorough evaluation of the model, as it tests the model on each individual sample.

Key Points:

- LOOCV can be computationally expensive, especially with large datasets, since it requires training the model as many times as there are samples.
- It provides an unbiased estimate of the model's performance since each sample is used for testing once.

D. Leave-P-Out Cross-Validation (LPO-CV) :

Leave-P-Out Cross-Validation involves leaving out P samples from the dataset as the test set while using the remaining samples for training. This process is repeated for all possible combinations of P samples.

Formula:

1. **Number of Combinations:** The total number of ways to choose P samples from N samples is given by the combination formula:

$$C(N,P) = N!/P!(N-P)!$$

2. **Process:** For each combination of PPP samples:

Training Set: Use N-P samples.

Test Set: Use the P samples left out.

3. **Performance Calculation:** After evaluating the model for each combination, average the performance metrics:

$$\text{Average Performance} = 1/C(N,P) \sum_{i=1}^C C(N,P) \text{Performance of } i$$

Example Calculation

Dataset: N=6 samples (A, B, C, D, E, F)

Leave 2 Out: P=2

1. **Calculate Combinations:**

$$C(6,2) = 6!/2!(6-2)! = 6 \times 5 / 2 \times 1 = 15$$

2. **Model Evaluation:**

- Evaluate the model on each of the 15 combinations of 2 samples.
- Suppose the performance for each combination is recorded.

3. **Average Performance:**

$$\text{Average Performance} = 1/15 \sum_{i=1}^{15} \text{Performance}_i$$

Summary

- **Total Combinations:** $C(6,2)=15$.
- Evaluate the model for each combination and calculate the average performance.

4. Resampling Methods

a. Bootstrapping:

- This method involves randomly selecting samples from the dataset with replacement (so the same sample can be picked multiple times).

- **Example:** If your dataset has 100 samples, you can create several new datasets by picking 100 samples randomly, some of which may be the same as others.

b. Random Subsampling:

- This method randomly splits the data into training and testing sets multiple times, without ensuring all data points are used.
- **Example:** Select 70% of the data for training and 30% for testing, and repeat this process several times.

5. Advantages of Cross-Validation and Resampling

- **Reliable Estimates:** These methods give a more accurate picture of how the model will perform.
- **Better Use of Data:** Especially with small datasets, they allow for more efficient training and testing.

6. Limitations

- **Time-Consuming:** Some methods, like LOOCV, require a lot of computation, which can take time.
- **Variability:** Different splits can lead to different performance results, making it harder to interpret.

Measuring classifier performance:

Measuring classifier performance is essential to understand how well your model is doing. Here are some common metrics and methods used to evaluate classifiers:

1. Accuracy

- **Definition:** The ratio of correctly predicted samples to the total samples.
- **Formula:** $\text{Accuracy} = \frac{\text{True Positives} + \text{True Negatives}}{\text{Total Samples}}$

2. Precision

- **Definition:** The ratio of correctly predicted positive samples to the total predicted positives.
- **Formula:** $\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$

3. Recall (Sensitivity)

- **Definition:** The ratio of correctly predicted positive samples to all actual positives.
- **Formula:** $\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$

4. F1 Score

- **Definition:** The harmonic mean of precision and recall, providing a balance between the two.

- **Formula:** $F1\ Score = 2 \times Precision \times Recall / Precision + Recall$

5. Confusion Matrix

- **Definition:** A table used to describe the performance of a classification model by showing true vs. predicted classifications.
- **Structure:**

	Predicted	
	Positive	Negative
Positive	TP	FN
Negative	FP	TN

6. ROC Curve (Receiver Operating Characteristic)

- **Definition:** A graphical representation of the trade-off between true positive rate (recall) and false positive rate at various threshold settings.
- **AUC (Area Under Curve):** Measures the overall performance of a model; a higher AUC indicates better performance.

7. Log Loss

- **Definition:** Measures the performance of a classification model whose output is a probability value between 0 and 1.
- **Formula:** $Log\ Loss = -1/N \sum_{i=1}^N [y_i \log(\pi_i) + (1-y_i) \log(1-\pi_i)]$

Where y_i is the true label and π_i is the predicted probability.

Summary

Choosing the right metric depends on the specific problem, especially in cases of class imbalance. For example, accuracy might be misleading when one class significantly outnumbers another. Metrics like precision, recall, and the F1 score often provide a more comprehensive picture of classifier performance.

K-fold CV Performance metrics-MSE, accuracy, confusion matrix, precision, recall, F1-score

K-Fold Cross-Validation Process

1. Split the Dataset:

- Divide the dataset into K folds.
- For each fold:
 - Use K-1 folds for training and 1 fold for testing.

2. Train and Evaluate:

- Train the model on the training set and evaluate it on the test fold.
- Record the performance metrics.

3. Aggregate Results:

- Average the metrics across all K folds to get the overall performance.

Performance Metrics

1. Mean Squared Error (MSE) (for regression tasks)

- **Definition:** Measures the average squared difference between predicted and actual values.
- **Formula:**

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

知乎 @新道Jdon

- **Interpretation:** Lower values indicate better model performance.

2. Accuracy (for classification tasks)

- **Definition:** The ratio of correctly predicted instances to the total instances.
- **Formula:** Accuracy = TP + TN / Total Samples
- **Interpretation:** Higher accuracy indicates better performance, but it can be misleading in imbalanced datasets.

3. Confusion Matrix (for classification tasks)

- **Structure:**

Predicted

| Positive | Negative

Positive | TP | FN

Negative | FP | TN

- **Definitions:**
 - **TP:** True Positives
 - **TN:** True Negatives
 - **FP:** False Positives

- **FN: False Negatives**

4. Precision

- **Definition:** The ratio of correctly predicted positive observations to the total predicted positives.
- **Formula:** $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$
- **Interpretation:** High precision indicates that fewer false positives were found.

5. Recall (Sensitivity)

- **Definition:** The ratio of correctly predicted positive observations to all actual positives.
- **Formula:** $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$
- **Interpretation:** High recall indicates that most actual positives were correctly identified.

6. F1 Score

- **Definition:** The harmonic mean of precision and recall.
- **Formula:** $\text{F1 Score} = 2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$
- **Interpretation:** The F1 score balances precision and recall; it's useful when you need a single metric to assess performance.

Linear Regression:

Example: Predicting Exam Scores

Dataset: Suppose you have a small dataset showing the number of study hours and the corresponding exam scores for five students:

Study Hours Exam Score

1	50
2	60
3	70
4	80
5	90

Step 1: Calculate the Means

1. Mean of Study Hours ($\bar{x}$)

$$\bar{x} = (1+2+3+4+5)/5 = 15/5 = 3$$

2. Mean of Exam Scores ($\bar{y}$):

$$\bar{y} = 50 + 60 + 70 + 80 + 90 / 5 = 350 / 5 = 70$$

Step 2: Calculate β_1

Use the formula:

$$\beta_1 = \sum (x_i - \bar{x})(y_i - \bar{y}) / \sum (x_i - \bar{x})^2$$

Numerator Calculation:

For each student:

- $(1-3)(50-70) = (-2)(-20) = 40$
- $(2-3)(60-70) = (-1)(-10) = 10$
- $(3-3)(70-70) = (0)(0) = 0$
- $(4-3)(80-70) = (1)(10) = 10$
- $(5-3)(90-70) = (2)(20) = 40$

Sum of the numerator:

$$40 + 10 + 0 + 10 + 40 = 100$$

Denominator Calculation:

- $(1-3)^2 = (-2)^2 = 4$
- $(2-3)^2 = (-1)^2 = 1$
- $(3-3)^2 = 0^2 = 0$
- $(4-3)^2 = (1)^2 = 1$
- $(5-3)^2 = (2)^2 = 4$ # all 2 is square here.

Sum of the denominator:

$$4 + 1 + 0 + 1 + 4 = 10$$

Step 3: Calculate β_1

Now, substitute the values into the formula:

$$\beta_1 = 100 / 10 = 10$$

Step 4: Calculate β_0

Use the formula:

$$\beta_0 = \bar{y} - \beta_1 \bar{x}$$

$$\beta_0 = 70 - 10 \times 3 = 70 - 30 = 40$$

Step 5: Final Equation

Now we have both coefficients:

- $\beta_0=40$
- $\beta_1=10$

The equation of the regression line is:

$$\text{Exam Score}=40+10\times\text{Study Hours}$$

Step 6: Predictions

Using the equation, you can predict scores. For example:

- If a student studies for 6 hours:

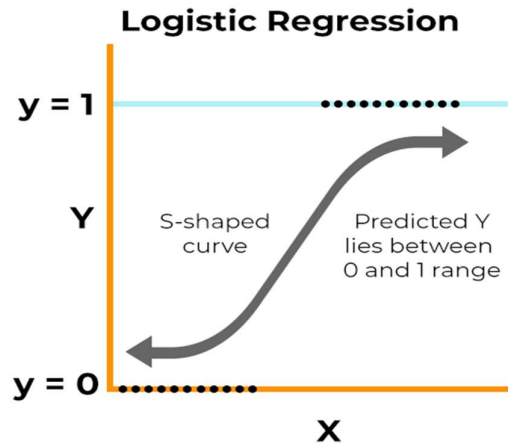
$$\text{Predicted Score}=40+10\times 6=40+60=100$$

Logistic Regression with Binomial & Multi class classification:

Definition: Logistic regression is a statistical method used for binary classification problems, where the outcome variable can take two possible outcomes (e.g., 0 and 1, Yes and No).

Logistic Function – Sigmoid Function

- The sigmoid function is a mathematical function used to map the predicted values to probabilities.
- It maps any real value into another value within a range of 0 and 1. The value of the logistic regression must be between 0 and 1, which cannot go beyond this limit, so it forms a curve like the “S” form.
- The S-form curve is called the Sigmoid function or the logistic function.
- In logistic regression, we use the concept of the threshold value, which defines the probability of either 0 or 1. Such as values above the threshold value tends to 1, and a value below the threshold values tends to 0.



Types of Logistic Regression

On the basis of the categories, Logistic Regression can be classified into three types:

1. **Binomial:** In binomial Logistic regression, there can be only two possible types of the dependent variables, such as 0 or 1, Pass or Fail, etc.
2. **Multinomial:** In multinomial Logistic regression, there can be 3 or more possible unordered types of the dependent variable, such as “cat”, “dogs”, or “sheep”

1. Binomial Logistic Regression

Concept: In binomial logistic regression, we model the probability that a certain class label is present. The logistic function (sigmoid) is used to map predicted values to probabilities between 0 and 1.

Mathematical Formulation: The logistic regression model can be represented as:

$$P(Y=1|X)=1/1+e^{-(\beta_0+\beta_1X)}$$

Where:

- $P(Y=1|X)$ is the probability of the event occurring (e.g., class 1).
- β_0 is the intercept.
- β_1 is the coefficient for the predictor X .
- **Example: Predicting Pass/Fail**
- Suppose we want to predict whether a student passes (1) or fails (0) based on study hours.

Study Hours	Pass (1) / Fail (0)
1	0
2	0
3	1
4	1
5	1

Fitting the Model: Using logistic regression, we may find:

- $\beta_0 = -4$
- $\beta_1 = 1$

The model equation would be:

$$P(\text{Pass}=1|\text{Hours}) = 1 / 1 + e^{-(-4+1 \times \text{Hours})}$$

Prediction: To predict the probability of passing with 3 hours of study:

$$P(\text{Pass}|3) = 1 / 1 + e^{-(-4+1 \times 3)} \approx 0.95$$

This means there is a 95% chance the student will pass.

Multiclass Classification

Definition: Multiclass classification is a type of classification problem where an instance can belong to one of three or more classes. Unlike binary classification (which has two classes), multiclass classification deals with multiple possible outcomes.